DAT321/DIT843: Quality Assurance and Testing

Questions about the exam contact:
Examiner: Francisco Gomes, tel. 073 856 4439, francisco.gomes@cse.gu.se

- Francisco or Sabina will visit the exam halls around 15:00 and 16:30.

No aids are allowed for this exam.

The questions in this exam refer to the **ISO 25010:2011** that categorises four types of quality: process, internal, external and quality in use. The exam also covers the eight attributes for product quality.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write correct, clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge or your learning of the topic.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product or software development process related to the questions. Avoid circular definitions and vague / generic answers.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper. The exam is anonymous, so do not leave any information that can reveal your identity.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 84: 4 (Pass with credit)
- 85 - 100: 5 (Pass with distinction)

Question 1. [Total: 25 pts] Using the context below, answer the following questions:

You work for **CityMove**, a large-scale urban mobility platform used daily by millions of users to plan trips, book rides, and navigate cities using different transport modes. The system consists of mobile apps, backend services, third-party integrations, and continuous deployment pipelines. Below is a list of features of the system. The system **shall**:

- **F1.** Maintain continuous route guidance when transitions occur between different navigation modes or map data sources during an ongoing trip.
- **F2.** Continue providing navigation support during temporary network degradation, such as high-traffic rush hours, without interrupting the active route guidance.
- **F3.** Collect location data in the background while limiting impact on device battery and responsiveness.
- **F4.** Be updated frequently with new functionality and changes to existing routing behavior.
- **F5.** Integrate real-time data from external transport and mapping service providers.
- **F6.** Operate on a wide range of smartphones and tablets, across multiple operating system versions, with varying hardware capabilities.
- **F7.** Adapt its user interface to support users with different accessibility needs without manual configuration.
- **F8.** Provide post-trip feedback about travel efficiency and environmental impact.
- **F9.** Validate changes to routing logic using automated tests before deployment to production.
- **F10.** Gradually enable new routing features for selected groups of users while monitoring system behavior.

Note: For the questions below, you are allowed to add features or decisions about the software development process (e.g., team size, technologies, etc.). However, any assumption related to your answer: (i) must be clearly stated, and (ii) cannot contradict any of the points above.

a. **[10 pts]** Choose **two of the features** and, for each feature: (i) identify **one relevant software product quality attribute** from ISO 25010, (ii) explain how this same attribute can be evaluated from the perspective of **one of the following types of quality***:* Process Quality, Internal Quality, External Quality, or Quality in Use*,* and (iii) propose **one concrete and measurable indicator** to quantify that attribute from the chosen quality perspective.

Your answer must clearly justify the chosen quality type and explain why the proposed measure is appropriate.

+5 per feature, where: +1 attribute, +2 quality type, +2 measure.

Below I listed a few varied examples of features in different types of qualities and measures that are acceptable answers.

F2. The system shall continue providing navigation support during temporary network degradation, such as high-traffic rush hours, without interrupting the active route guidance.
(i) Reliability because the core risk is interruption or failure of navigation (functionality) under specific time or adverse conditions.
(ii) External Quality because this is evaluated by observing runtime behavior in realistic conditions (degraded network), not by inspecting the code or the process.
(iii) Measure: Avaialbility of route-guidance during degraded network, where we measure the uptime versus the downtime of the service with a specific goal (e.g., 99.5%.)

F3. The system shall collect location data in the background while limiting impact on device battery and responsiveness.
(i) Performance Efficiency because the requirement is about resource use (battery, CPU) and responsiveness.
(ii) Even though performance efficiency is typically external quality, **we can use it for Internal Quality** measures. For insteance, we can evaluate it with artefact measurements such as profiling and instrumentation of the app, focusing on resource utilization linked to code and design decisions (e.g., algorithmic complexity).
(iii) Measure: Algorithmic complexity of functions related to data handling, where we use tools to identify whether the time and storage of the algorithms are not at exponential scale $O(2^n)$.

F9. The system shall validate changes to routing logic using automated tests before deployment to production.
(i) Maintainability because automated tests support safe modification, faster regression detection, and easier long-term evolution.
(ii) We can measure it to improve **Process Quality** by using this feature to describe how the team develops and releases software, so it is primarily evaluated through pipeline practices and process outcomes.
(iii) We can measure the **change failure rate in** production as the % of deployments that result in a production incident, **hotfix, or rollback within 24 hours**.

F10. The system shall gradually enable new routing features for selected groups of users while monitoring system behavior.
(i) ISO 25010 quality attribute: Reliability
Reason: Gradual rollout is used to reduce the risk of widespread failures and to detect instability early.

(ii) Quality type chosen: Process Quality
Reason: This is a release and operational practice. It is evaluated by how rollout is managed and how quickly issues are detected and contained.
(iii) Measure: Mean time to detect and stop a bad rollout (MTTD + mitigation)

- Definition: time from first abnormal signal (error rate, crash rate, latency spike) to automatic or manual rollback or feature-flag disablement.
- Example target: ≤ 10 minutes.

F1. The system shall maintain continuous route guidance when transitions occur between different navigation modes or map data sources during an ongoing trip.
(i) Performance Efficiency because Mode and data-source transitions must not introduce delays that affect navigation continuity.
(ii) This can be evaluated as **External Quality** by observing system behavior at runtime, specifically how quickly the system responds during transitions experienced by users.
(iii) Measure: Latency as the time between initiating a navigation mode or map data-source transition and the system resuming active route guidance.

F4. The system shall be updated frequently with new functionality and changes to existing routing behavior.
(i) Maintainability because it conveys frequent changes increase the importance of modular, understandable, and modifiable code.
(ii) Internal Quality evaluated by inspecting code-level properties that influence how easily developers can change routing behavior.
(iii) Measure: Code churn in routing modules by looking at the number of lines added, modified, or deleted per release in routing-related components, normalized by module size.

(Alternative acceptable internal measures would include cyclomatic complexity or cognitive complexity of routing functions.)

b. [8 pts] Select **two other features** and, for each: describe a corresponding quality attribute, and a quality scenario. Clearly specify the (i) source, (ii) stimulus, (iii) artefact, (iv) environment, (v) response, and (vi) response measure.

Note: The features **must be different** than the ones chosen in Q1.a.

+4 per quality scenario. There should be cohesion between the elements of the scenario, and the quality attribute chosen.

Feature: F2 (Reliability)
Quality scenario:

- **Source:** Mobile user navigating
- **Stimulus:** Network connectivity degrades due to congestion
- **Artefact:** Navigation and route-guidance service

- **Environment:** Urban area, peak traffic hours, mobile network under high load
- **Response:** Navigation guidance continues without crashing, or resetting the route
- **Response measure:** Route guidance remains uninterrupted in **≥ 99.5%** of sessions where network degradation lasts at least 60 seconds

---

Feature: F3. (Performance Efficiency)
Quality scenario:

- **Source:** User running CityMove **in the background**
- **Stimulus:** Continuous background location tracking during a 30-minute trip
- **Artefact:** Mobile app background services and location-tracking component
- **Environment:** Battery level between 20–50%, CityMove is running in the backgroun, other apps are **running in the foreground**
- **Response:** App tracks location without noticeable slowdown or unusual battery drain
- **Response measure:** Battery consumption attributable to CityMove is **≤ 4% per hour**, and foreground app interaction latency increases by **≤ 100 ms**

Feature: F7. (Usability)
Quality scenario:

- **Source:** User with visual impairment relying on screen reader functionality
- **Stimulus:** User launches the app and starts navigation for the first time
- **Artefact:** Mobile app user interface and accessibility layer
- **Environment:** Smartphone with system-level accessibility features enabled
- **Response:** Interface elements are readable, navigable, and correctly announced without additional user setup
- **Response measure:** User successfully completes route setup and navigation start on first attempt in **≤ 2 minutes**, with **zero accessibility-related errors**

Feature: F5. (Compatibility)
Quality scenario:

- **Source:** CityMove navigation service
- **Stimulus:** CityMove requests real-time schedule data from a **public transport API**
- **Artefact:** Data integration and routing coordination layer
- **Environment:** System during weekday rush hours with high request volume
- **Response:** CityMove successfully combines public transport data and map data to generate valid routes without conflicts, inconsistencies, or missing connections
- **Response measure: ≥ 99.9%** of requests correctly include public transport segments data, with **zero failures caused by data incompatibilities.**

c. **[7 pts]** CityMove wants to promote sustainability in the product that we are creating. Using the Sustainability as a quality attribute, you must (i) **choose or suggest two functionalities** for the product and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the corresponding dimension of sustainability in which those effects.

Note: You can choose **any** of the 10 listed features (even if you used it for Q1.a and Q1.b), or create one yourself. If creating a new feature, use the template: "The system shall.."

+3 per feature. +1 for overall cohesion of the effects:

F8. The system shall provide post-trip feedback about travel efficiency and environmental impact.

- **Immediate effect:** Users become aware of the environmental impact of their trip, such as CO2 ommisions (Individual).
- **Enabling effect:** Users can compare trips over time and adjust future travel behavior (Individual).
- **Structural effect:** Aggregated behavior changes reduce congestion and emissions at city scale (Environmental).

F10. The system shall gradually enable new routing features for selected groups of users while monitoring system behavior.

- **Immediate effect:** Limits large-scale failures by containing faulty releases (Technical).
- **Enabling effect:** Supports data-driven refinement before full rollout (Technical, Economic).
- **Structural effect:** Establishes long-term practices for responsible and stable system evolution (Technical, Community).

F11. *The system shall* adapt its user interface automatically to support users with different accessibility needs without manual configuration.
Dimension: Individual

- **Immediate effect:** Users with accessibility needs can successfully use navigation features without additional setup or assistance (Individual).
- **Enabling effect:** Consistent accessible interaction reduces frustration and increases user confidence and autonomy when planning and navigating trips (Individual).
- **Structural effect:** Over time, inclusive design becomes a baseline expectation for digital mobility services, reducing exclusion of users with disabilities (Community).

F12. *The system shall* provide aggregated, anonymized mobility usage insights to city stakeholders to support urban planning and public transport improvements.
Dimension: Community

- **Immediate effect:** City planners gain visibility into mobility patterns and underserved areas based on real usage data (Community).
- **Enabling effect:** Data-informed decisions enable targeted improvements to infrastructure, public transport routes, and accessibility services (Community).
- **Structural effect:** Long-term integration of shared mobility insights supports more inclusive, efficient, and resilient urban transport systems (Community).

Question 2. [Total: 25 pts] Based on your knowledge of Property-based testing (PBT), and Black-box testing, answer the questions below. We listed also some examples of methods below that can be used when writing your properties.

`sum(int[]): int`	Returns the sum of the integers in the specified array.
`length(int[]): int`	Returns the number of elements in the array.
`sort(int[]): int[]`	Returns the array with its values sorted in an ascending order.
`reverse(int[]): int[]`	Returns the array where its values are in reversed positions.
`max(int[]): int`	Returns the highest integer value in the array.
`min(int[]): int`	Returns the lowest integer value in the array.

Specification: The function filterScoresInRange(List<Integer> scores, int lower, int higher) receives (i) a list of integers scores, (ii) an integer lower, and (iii) an integer higher. The function returns a **new list** containing only the scores from the original list such that **lower <= score <= higher**. The **order of the returned scores must be the same** as in the original list (stable filtering), and repeated values must be preserved.

If list of scores is **empty**, the function must return an empty list. If **list of scores is null**, the function must throw an IllegalArgumentException with the message "Invalid list.". If **higher < lower**, the function must throw an IllegalArgumentException with the message "Invalid range.". Below, we included 10 examples of test cases for the code above.

T1	`filterScoresInRange([10, 5, 7, 3, 8], 5, 8) --> [5, 7, 8]`
T2	`filterScoresInRange([4, 4, 5, 4, 6], 4, 4) --> [4, 4, 4]`
T3	`filterScoresInRange([1, 2, 3, 4], 1, 4) --> [1, 2, 3, 4]`
T4	`filterScoresInRange([1, 2, 3], 10, 20) --> []`
T5	`filterScoresInRange([], 0, 5) --> []`
T6	`filterScoresInRange(null, 0, 5) --> Exception: "Invalid list."`
T7	`filterScoresInRange([1, 2, 3], 5, 4) --> Exception: "Invalid range."`
T8	`filterScoresInRange([-5, -2, 0, 3, 7], -2, 3) --> [-2, 0, 3]`
T9	`filterScoresInRange([-10, -3, -3, -1], -3, -1) --> [-3, -3, -1]`
T10	`filterScoresInRange([-8, -2, 1, 2], -10, -3) --> [-8]`

Note: You can find the code for this SUT in Question 3.

a. **[7 pts]** Create **exactly three** properties for the SUT above. You can write the properties using natural language, mathematical notation, or pseudocode. Feel free to use any of the support functions for arrays of integers listed in the beginning of this question. Your properties will be evaluated based on how diverse they are.

+2 per property, +1 for variety. Some examples of properties are:

- P1: For any high and low, scores == null: filter(scores, high, low) == Exception.
- P2: For any scores != null, low >= high: filter(scores, high, low) == Exception
- P3: For scores with the same element x AND low <= x <= high: filter(scores, high, low) == scores.
- P4: For any scores where min(scores) >= low AND max(scores) <= high: filter(scores, high, low) == scores

Note: Some tips for other properties will be to reduce the set of inputs regarding score values bound by high and low.

b. [8 pts] Create **exactly four equivalent partition** classes for the SUT above.

+2 per partition. Clarify the set of input and the observed behaviour in each partition. Some examples of partitions are:

EP1: The list of scores is null, for any value of the lower and higher bounds. Expected behaviour: the function throws an exception indicating an **invalid list**.

EP2: The list of scores is not null and the higher bound is smaller than the lower bound, for any contents of the list. Expected behaviour: the function throws an exception indicating an **invalid range**.

EP3: The list of scores is empty and the lower bound is smaller than or equal to the higher bound. Expected behaviour: the function returns an empty list.

EP4: The list of scores is not empty and all elements of the lsit are the same and within the high and score. Expected behaviour: the function returns the same list given as input.

EP5: The list of scores is not empty and the highest score is less than low, and high is greater than low. Expected behaviour is an empty array.

c. [10 pts] Using the partition classes that you defined above, write test cases that reveal **exactly four boundaries**. For each boundary, you must indicate (i) the corresponding equivalent partition classes, and whether they are a VV, EE, or VE boundary.

+2 per boundary. +2 for the variety of the validity class. Some examples of boundaries:

Boundary 1: VE
TC1 (EP1): Input: scores = null, lower = 0, higher = 5. Expected: Exception "Invalid list."

TC2 (EP4): Input: scores = [0], lower = 0, higher = 5. Expected: [0]

Boundary 2: VE
TC3 (EP5): Input: scores = [1, 2, 3], lower = 5, higher = 5. Expected: []
TC4 (EP2): Input: scores = [1, 2, 3], lower = 5, higher = 4. Expected: Exception "Invalid range."

Boundary 3: VV
TC5 (EP3): Input: scores = [], lower = 0, higher = 5. Expected: []
TC6 (EP4): Input: scores = [0], lower = 0, higher = 5. Expected: [0]

Boundary 4: EE
TC1 (EP1): Input: scores = null, lower = 0, higher = 5. Expected: Exception "Invalid list."
TC4 (EP2): Input: scores = [1], lower = 5, higher = 0. Expected: Exception "Invalid range."

Question 3. [Total: 25 pts] Based on your knowledge of coverage criteria, white-box testing, and mutation testing, answer the questions below. The code below is the Java implementation of the filterScoresInRange function specified in Question 2.

```
1.    import java.util.ArrayList;
2.    import java.util.List;
3.
4.    public class ScoreFilter {
5.
6.      public static List<Integer> filterScoresInRange(List<Integer> scores,
7.                                                      int lower, int higher) {
8.        // Invalid list
9.        if (scores == null) { // Branch A
10.         throw new IllegalArgumentException("Invalid list.");
11.
12.       } else if (higher < lower) { // Invalid range (Branch B)
13.         throw new IllegalArgumentException("Invalid range.");
14.
15.       }
16.
17.       // Create a new list for the result (do not modify the input list)
18.       List<Integer> result = new ArrayList<>();
19.
20.       // Stable filtering: keep original order
21        for (int i = 0; i < scores.size(); i++) { // Loop Condition (Branch C)
22.         int score = scores.get(i); // assume list has no null elements
23.
24.         if (score >= lower && score <= higher) { // Branch D
25.           result.add(score);
26.         }
27.       }
28.
29.       return result;
30.     }
```

```
31.  }
```

Consider that three mutants are generated for the code below.

- **M1 - line 12:** if (higher < lower) → if (higher <= lower)
- **M2 - line 24:** score >= lower → score > lower
- **M3 - line 24:** score <= higher → score < higher

Consider the test suite **TS = {T1, T2, T3, T4}** (from the tests in Question 2). A coverage tool shows the coverage metrics: Statement coverage: 100%, Branch coverage: 62.5%.

a. [7 pts] Using your knowledge of mutation testing, (i) for each mutant (M1–M3), explain (i) whether the mutant is **killed or survives by indicating both** the actual and expected output, and (ii) the corresponding mutation coverage for TS.

Recall that the mutation coverage is obtained by dividing the number of mutants killed by the total number of mutants.

Begin by doing a mutation matrix. In the answer, it is not required to list all tests that failed per mutant, but the complete matrix is below.

	M1	**M2**	**M3**
T1	Survived	Killed	Killed
T2	Killed	Killed	Killed
T3	Survived	Killed	Killed
T4	Survived	Survived	Survived

+5 pts for mutation results. +2 pts for coverage value.

M1:

- T1: Expected = [5,7,8] Actual: [5,7,8].
- T2: Expected = [4,4,4]. Actual: Exception **(Killed)**
- T3: Expected = [1,2,3,4]. Actual: [1,2,3,4]
- T4: Expected = []. Actual = []

M2:

- T1: Expected = [5,7,8] Actual: [7,8] **(Killed)**
- T2: Expected = [4,4,4]. Actual: [] **(Killed)**
- T3: Expected = [1,2,3,4]. Actual: [2,3,4] **(Killed)**
- T4: Expected = []. Actual = []

M3:

- T1: Expected = [5,7,8] Actual: [5,7] **(Killed)**
- T2: Expected = [4,4,4]. Actual: [] **(Killed)**
- T3: Expected = [1,2,3,4]. Actual: [1,2,3] **(Killed)**
- T4: Expected = []. Actual = []

Since all mutants are killed, the mutation coverage is **3 / 3 = 100%**.

b. **[10 pts]** Using all the coverage measures presented, do you think that TS is a **poor** test suite? Explain your answer by relating **mutation coverage**, **statement coverage, branch coverage**, and **the intent of the specification.**

+2 Yes, it is poor in terms of SUT behaviour. +4 explanation for code cov.. +4 explanation for mutation cov.

TS is a poor test suite, because it does not cover many cases, particularly for the invalid behavior (Exceptions). TS is an **incomplete** test suite.

This lacking behavior coverage is reflected in the **relatively low branch coverage** where the **first two branches are never evaluated to 'true'** (i.e., they do not throw the exception). Moreover, there is **no case for skipping the loop** (even though the branch of terminating the loop is covered).

While the mutation coverage is high (100%), **all mutants belong to the same operator**. This means that the test suite is sensitive to specific types of faults (**relational**). Therefore, we should try more varied mutants.

c. **[8 pts]** Suggest **two additional test cases** that would improve the quality of TS. Your answer must: (i) specify the input and expected output, and (ii) explain **which coverage or mutation weakness it addresses**.

Note: You can create your own test cases, or choose from the 10 tests in Question 2.

+4 per test case (+1 values, +3 explanation). Some examples of tests that can help the missing branches or loop case are T5, T6, T7

Question 4 [Total: 25 pts] Based on your knowledge of regression testing and test suite optimisation techniques, answer the questions below.

The table below shows the traceability between a test cases and corresponding system requirements (R1 – R6). Each test case is executed manually, and can take several minutes to finish. Due to these high costs, executing all test cases is often infeasible.

Test Case	R1	R2	R3	R4	R5	R6
T1	X	X	X			
T2		X	X			

T3	X		X			
T4				X	X	
T5					X	
T6					X	X
T7				X		

a. **[7 pts]** Your colleague Mary says that the test suite $\boldsymbol{TS} = \{\boldsymbol{T1}, \boldsymbol{T3}, \boldsymbol{T4}, \boldsymbol{T6}\}$ is a valid minimized test suite for this problem. Do you agree or disagree with Mary? Justify your answer.

+1 quality of explanation. +3 justify that T3 is 1-to-1 redundant to T1. +3 identify a test suite that provides better minimisation **TS = {T1, T4, T6}**

b. **[10 pts]** The table below shows information from faults identified in past executions of the test suite. Your co-workers suggested three different prioritised test suites to be executed.

- TA = [T1, T2, T3, T4, T7, T6, T5]
- TB = [T3, T4, T2, T1, T6, T5, T7]
- TC = [T2, T6, T7, T1, T5, T3, T4]

Test Case	F1	F2	F3	F4	F5	F6	F7	F8
T1	X	X	X					
T2	X							
T3				X	X	X		
T4							X	
T5								X
T6		X	X					
T7					X			X

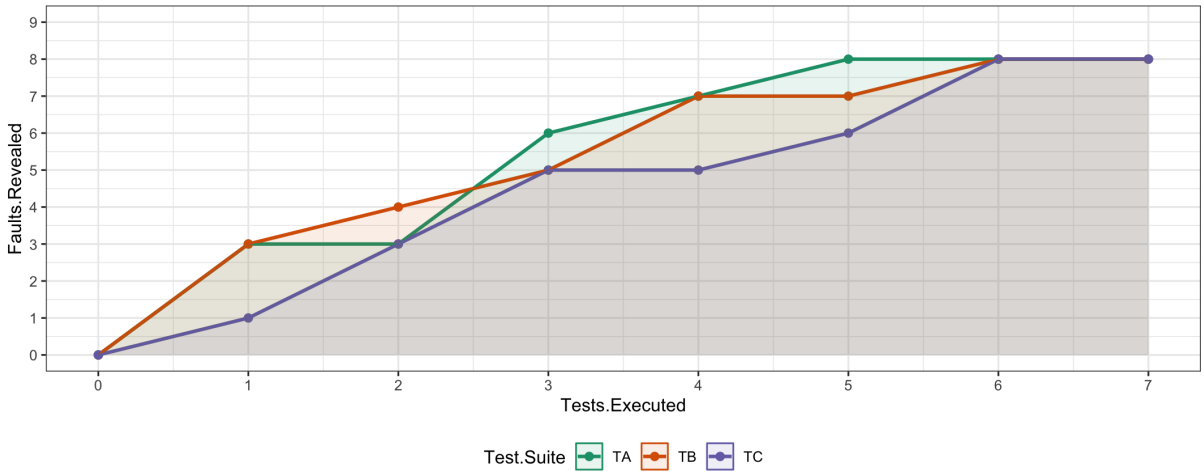
Using the concept of APFD (average percentage of faults detected), evaluate the three prioritized test suites TA, TB, and TC. Justify your evaluation by highlighting how cost-effective the different test suites are.

Note: You don't need to remember the formula or calculate the specific APFD value. You can answer using the intuition behind the APFD measure, or represent the effectiveness of the test suites visually (e.g., via an area under the curve).

+2 plot
+2 explain that TC is worse
+3 explain that TA is better for more than half the tests
+3 explain that TB is better for less than half of the tests.

Example of an answer:

Here is the plot for the area under the curve for the number of faults revealed (y-axis) in relation to the number of tests executed (x-axis)



As we can see, TC is the least cost-effective technique as it does not reveal any more faults than the other two at any point. However, TA is a better test suite, specially if we can any tests as it reveals more faults towards the end (after 3 tests executed). Note that for very prohibitive scenarios (executing less than half of the tests, TB reveals more faults).

c. **[8 pts]** Diversity can be used as a cost-effective prioritisation criteria. Explain (i) how we can measure diversity of a test suite; (ii) one advantage of diversity-based test prioritisation, (iii) one disadvantage of diversity-based test prioritisation.

(i) **+2 pts:** Diversity can be measured by **encoding the diverisity information** based on aspects such as input values, executed code elements, covered requirements, or detected faults. Then, we choose a **distance or similarity metric** between test cases, such set-based metrics such as Jaccard distance or structural metrics based on coverage vectors. The diversity of a test suite is then computed by **aggregating pairwise distances** (e.g., in a matrix), for example by maximizing the average distance between selected tests or by using a max-min strategy.

(ii) **+3 pts:** Explain some advantages such as: DBT can be used in **any artefact** if we can encode the diveristy value correctly; it ensures **coverage of diverse behaviour** to minimise risks of faults “slipping”, useful when **historical data is**

missing.

(iii) **+3 pts:** Some advantages are: DBT is **computationallly expensive** for larger test suites, does **not account for failure history or severity**, by default it **ignores test execution cost and constraints**.